

Project: Food Remedy API

Ticket: RE-018 – Local Food Database Update and Storage Feasibility

Author: Gurkirat Singh Chahal

Student ID: 223936693

Date: 27/08/2026

Table of Contents

1. Introduction
2. Current Food Remedy Storage Architecture
3. Local Storage Options and Storage Impact
4. Updating Local Product Data
5. Update Frequency and Data Freshness
6. Reliability and Synchronisation Risks
7. Recommended Architecture for Food Remedy
8. Conclusion
9. References

1. Introduction

To give users information about ingredients, nutrition, and allergies, Food Remedy uses food and product data. Expo SQLite is currently utilised for specific types of local application data, whereas Firestore is meant to serve as the main global product catalogue.

There are now about 61,373 Australian product records in the repository, which correspond to about 61,365 distinct barcodes. Before being seeded into Firestore, product data is handled using a mostly batch-based method that includes roughly monthly data pulls, cleaning, and enrichment. Open Food Facts is the primary source of product information.

In order to enhance offline functionality, application performance, and minimise needless cloud processes, this study explores if Food Remedy could realistically store more product information locally. Before suggesting a strategy appropriate for the current Food Remedy architecture, it evaluates local storage approaches, storage requirements, updating techniques, data freshness, and synchronisation issues.

2. Current Food Remedy Storage Architecture

Both local and online storage are already used by Food Remedy. While Expo SQLite offers persistent on-device storage, Firestore is meant to supply the shared product catalogue. It doesn't seem to be a part of the current startup process, despite the existence of code for

loading a wider product catalogue into SQLite. As a result, Food Remedy's offline product catalogue is not yet fully synced.

Table 1. Current Storage Environment

Area	Current Approach	Main Consideration
Product catalogue	Firestore	Connectivity may be required for uncached products
Product source	Open Food Facts	Product information can change
Dataset	~61,365 unique barcodes	Future catalogue size is unknown
Local storage	Expo SQLite	Complete catalogue is not currently synchronised
Product updates	Monthly pull + ad-hoc updates	No automated complete local sync identified
Favourites/history	Primarily local	Cached product information may become outdated
Shopping lists	Local/cloud sync	Failed writes require stronger retry handling

Expo SQLite can be used to expand Food Remedy's local product-storage capabilities without the need for additional database technology because it already offers persistent structured storage beyond programme restarts.

3. Local Storage Options and Storage Impact

There are a number of options, from keeping the complete product library locally to staying mostly cloud-based.

Table 2. Local Storage Options

Approach	Offline Capability	Storage	Complexity	Suitability
Cloud-focused	Low	Very low	Low	Moderate
Firestore cache	Moderate	Low	Low	Useful supporting mechanism
Selective SQLite cache	High for cached products	Low–moderate	Moderate	High
Full SQLite catalogue	Very high	High	High	Possible but currently unnecessary

Firestore + SQLite hybrid	High	Moderate	Moderate	Recommended
---------------------------	------	----------	----------	--------------------

This is not the same as purposefully keeping an entire offline product catalogue, but Firestore's offline capabilities can make previously accessed content accessible while offline.

Because records vary greatly depending on the fields maintained, the present catalogue size alone is also insufficient to determine an exact storage requirement. Compared to replicating every Open Food Facts field that is accessible, a compact record that only contains the essential product information would use significantly less storage.

Therefore, Food Remedy should refrain from including the entire collection with each application version. For food information to change without necessitating a new application version, product data should continue to be independently updateable.

4. Updating Local Product Data

Every time the catalogue is updated, local product information must stay up to date without requiring an application update.

There are three primary methods.

Table 3. Product Update Approaches

Factor	Full Update	Incremental Update	Hybrid Sync
Method	Replace complete database	Download changed records	Refresh selected local records
Data transfer	High	Low	Low–moderate
Complexity	Low–moderate	Moderate	Moderate
Freshness	Schedule dependent	Good	Good
Best use	Bootstrap/recovery	Routine catalogue changes	Recently accessed products
Food Remedy suitability	Moderate	High	High

For initial setup, significant schema modifications, or recovery, a complete database replacement is still helpful. However, it would be ineffective to download over 61,000 records continually when just few have changed.

Incremental updates are better for regular functioning. Metadata like barcode, updatedAt, datasetVersion, and syncedAt may be preserved in local records. Only records that have changed since the device's most recent successful update could then be downloaded by Food Remedy.

When a product is accessed online, a hybrid process might additionally refresh SQLite:

SQLite check → Show data that is available → Verify the official version while online → Refresh SQLite if it has been updated

This keeps Firestore as the authoritative source while offering quick local access.

5. Update Frequency and Data Freshness

The current Food Remedy pipeline pulls complete Open Food Facts about once a month, adding changes as needed. As a result, mobile devices do not have to download the full inventory on a regular basis.

A smaller dataset manifest with data like this would be a preferable strategy.

version of the dataset;

- The date of publication
- version of the schema;
- the number of products; and
- If a complete rebuild is necessary.

There is no need to update the catalogue if the local version is up to date. Only the required modifications could be downloaded if a more recent validated dataset is available.

Additionally, Food Remedy needs to differentiate between product information that is current, cached, old, and unknown-freshness. Because product formulas might change, this is especially crucial for information about ingredients and allergens.

Stale offline data, for instance, could be shown using:

"Displaying information about saved products. To confirm the most recent product details, connect to the internet.

6. Reliability and Synchronisation Risks

Increasing local storage creates new dependability challenges that must be addressed.

Table 4. Main Risks and Controls

Risk	Potential Problem	Recommended Control
Stale information	Outdated product details	Freshness/version metadata
Failed update	Partial local catalogue	Transactions and retry
Version conflict	SQLite differs from Firestore	Treat authoritative dataset as source of truth

Duplicate products	Multiple records per barcode	Unique identifier constraints
Storage growth	Excessive cache size	Cache limits/cleanup
Database corruption	Offline data unavailable	Rebuild/recovery process
Failed user-data sync	Local changes not uploaded	Durable retry/outbox
Schema changes	Old app cannot read new records	Schema versioning/migrations

Because the current project lacks a comprehensive local catalogue reconciliation policy and already exhibits some inconsistent synchronising behaviour, these concerns are particularly pertinent.

As a result, explicit guidelines for freshness, versioning, retries, and conflict resolution should go hand in hand with SQLite's expansion.

7. Recommended Architecture for Food Remedy

A hybrid Firestore + Expo SQLite architecture is the best strategy.

SQLite should offer restricted offline access to pertinent product data, while Firestore should continue to be the official shared product catalogue.

Suggested Data Flow

Cleaning/Validation → Approved Dataset → Firestore → Mobile App → SQLite Cache → Open Food Facts

Table 5. Recommended Actions

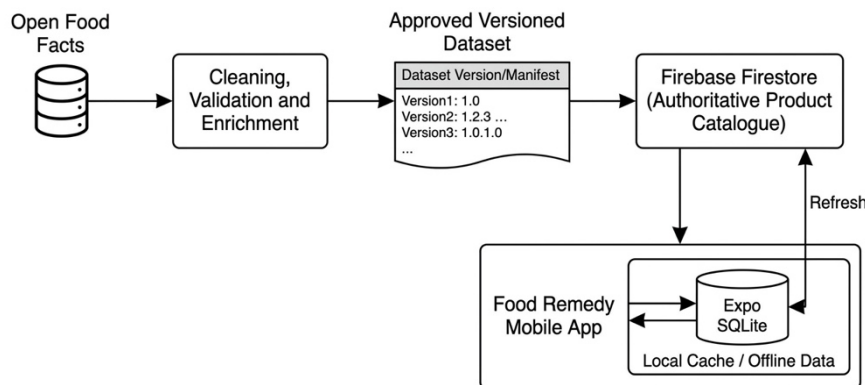
Priority	Recommendation
High	Keep Firestore as authoritative product source
High	Continue using Expo SQLite for local structured storage
High	Cache recently scanned/viewed products
High	Store freshness and version metadata
High	Define Firestore-to-SQLite refresh rules
High	Add reliable retry handling for synchronised user data
High	Standardise PRODUCTS / products collection naming
Medium	Introduce incremental product updates
Medium	Introduce dataset/schema versioning
Medium	Clearly identify stale cached information

Medium	Define cache retention limits
Future	Consider full offline catalogue only if testing demonstrates a need

Instead than needing Food Remedy to update its current technology, this recommendation works with the current architecture. The hybrid strategy offered the best mix between offline availability, current data, device storage, cloud utilisation, and scalability, according to your initial assessment.

Visual – Food Remedy Hybrid Product Data Architecture

Food Remedy Hybrid Product Data Architecture



8. Conclusion

According to this study, Food Remedy can technically increase local product storage, however it is not yet required to save the complete product catalogue on every device.

The best strategy is to use Expo SQLite as a regulated local cache and offline layer while keeping Firestore as the official product catalogue. While incremental updates and version metadata can keep information reasonably current without constantly downloading the full catalogue, recently browsed products can be saved locally.

Initial setup, recovery, and significant schema modifications should be the primary uses for full database replacement. Incremental or on-demand synchronisation should be used for routine updates. To lower the hazards of outdated or inconsistent data, freshness indicators, retry mechanisms, transactions, schema versioning, and cache restrictions should all be utilised.

Overall, while maintaining compatibility with the project's current Firestore and Expo SQLite architecture, the hybrid method offers Food Remedy the best mix between offline

functionality, performance, stability, storage requirements, and less needless cloud activities.

References

- Expo. (2026a). SQLite. Expo Documentation.
<https://docs.expo.dev/versions/v54.0.0/sdk/sqlite/>
- Expo. (2026b). Assets. Expo Documentation.
<https://docs.expo.dev/develop/user-interface/assets/>
- Google. (2026a). Access data offline. Firebase Documentation.
<https://firebase.google.com/docs/firestore/manage-data/enable-offline>
- Google. (2026b). Usage and limits. Firebase Documentation.
<https://firebase.google.com/docs/firestore/quotas>